



Dataset Used: kc_house_data.csv

The dataset used is **KC House Data**, which contains housing details along with their corresponding prices.

It includes features such as bedrooms, bathrooms, square footage, floors, condition, grade, and year built.

The target variable is **house price**, which is a continuous value representing the cost of each house.

This dataset is commonly used for **regression problems**, as it involves predicting numerical values.

In this project, both **classification and regression algorithms** are applied to understand different machine learning approaches and compare their performance.

Goal: The main goal of this project is to apply and evaluate multiple machine learning algorithms on the KC housing dataset.

It aims to analyze how different models perform using housing features such as area, bedrooms, and condition.

The project focuses on understanding the importance of data preprocessing and feature scaling in improving model performance.

Another objective is to compare both classification and regression algorithms to identify the most effective approach.

Finally, it helps in determining which type of model is more suitable, concluding that regression is better for predicting continuous values like house prices.

Modules Used:

The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (`np.diff`).
- **Scikit-Learn:** Scikit-learn is the main machine learning library used in this project. It provides tools for data splitting, preprocessing, model building, and evaluation. All classification algorithms used in this project come from this library.
- **train_test_split:** This module is used to divide the dataset into training and testing sets. It ensures that the model is evaluated on unseen data. In this project, 80% data is used for training and 20% for testing.
- **StandardScaler:** StandardScaler is used for feature scaling. It standardizes features by converting them into a common scale. This improves model performance and prevents bias due to feature range differences.
-
- **Classification Algorithms (from Scikit-learn):**
 - SVC – Used for Support Vector Machine classification
 - LogisticRegression – Used for probability-based classification
 - GaussianNB – Used for probabilistic classification
 - DecisionTreeClassifier – Used for tree-based decisions
 - RandomForestClassifier – Used for ensemble learning
 - KNeighborsClassifier – Used for distance-based classification (KNN)
 - GradientBoostingClassifier – Used for sequential ensemble learning
- **Regression Algorithms (Scikit-learn)**
 - LinearRegression – Predicts continuous values
 - Ridge – Regularized linear regression
 - DecisionTreeRegressor – Tree-based regression
 - RandomForestRegressor – Ensemble regression
 - SVR – Support Vector Regression
 - GradientBoostingRegressor – Boosting-based regression
- **accuracy_score:** This module is used to evaluate the model performance. It calculates the accuracy of predictions made by each algorithm. Helps in comparing different models.
- **Evaluation Metrics**
 - accuracy_score – Used for classification model evaluation
 - r2_score – Used for regression model evaluation

Classification Algorithms Implemented in the Project

1. Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised learning algorithm used for classification tasks. It works by finding an optimal boundary that separates data into different categories with maximum margin. In this housing project, SVM attempts to classify houses into price-related categories using features such as bedrooms, square footage, and condition. After applying feature scaling, it analyzes patterns between housing features and output values. SVM performs well in high-dimensional data and can handle non-linear relationships using kernels. However, housing prices are continuous values, making classification less appropriate. It struggles when price ranges overlap significantly. Selecting the correct kernel and parameters is difficult. It is also computationally expensive for large datasets like KC housing data.

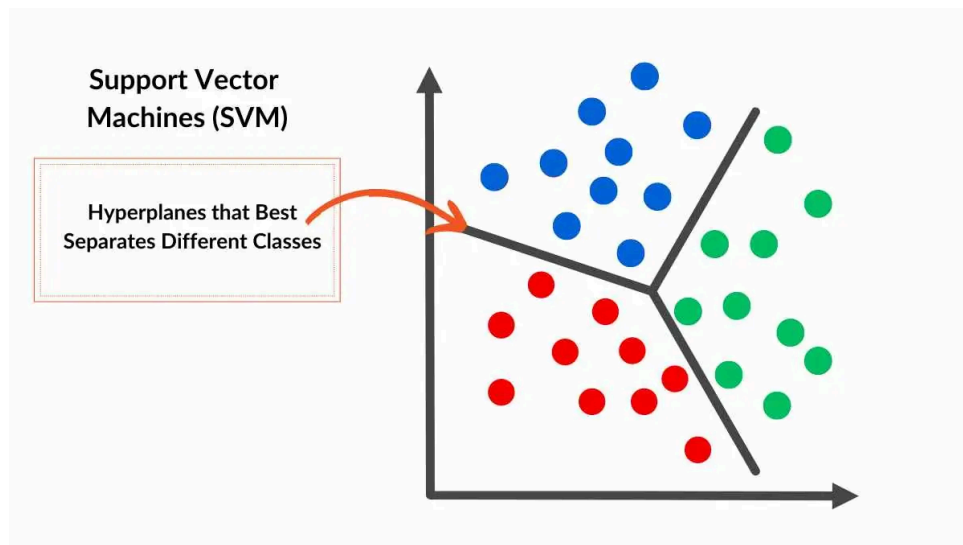


Figure 1.1: Explaining SVM Classifier

Limitations

- Support Vector Machine is powerful, but it becomes computationally expensive when applied to large datasets, especially with non-linear kernels like RBF. Training time increases significantly as the number of samples grows.
- Selecting the appropriate kernel function and hyperparameters (C, gamma) is challenging and requires careful tuning, often through trial-and-error or cross-validation.
- SVM does not perform well when the dataset contains significant noise or overlapping classes, as it tries to strictly separate classes using margins.
- It is also less interpretable, meaning it is difficult to understand how the model makes decisions compared to models like Decision Trees.
- Additionally, SVM requires proper feature scaling, and performance may degrade if features are not normalized.

2. Logistic Regression

Logistic Regression is a supervised classification algorithm that predicts probabilities and assigns outputs based on a threshold. In this project, it is used to classify house prices into categories based on features like area and number of rooms. It assumes a linear relationship between input features and the output. The model is simple, fast, and easy to interpret. It works well when the data is linearly separable. However, housing data often contains complex and non-linear relationships. Logistic Regression may oversimplify these patterns. It is sensitive to outliers such as luxury houses. It is not suitable for predicting exact price values.

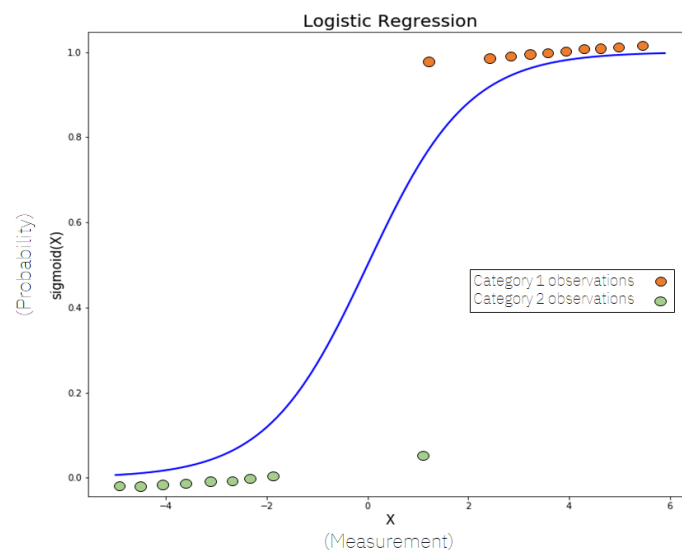


Figure 1.2: Explaining Logistic Regression

Limitations

- Logistic Regression assumes a linear relationship between independent variables and the log-odds of the dependent variable, which limits its performance on complex, non-linear datasets.
- It struggles when decision boundaries are not linear unless feature engineering or transformations are applied.
- The model is highly sensitive to outliers, which can significantly distort the decision boundary and affect predictions.
- Another limitation is multicollinearity, where highly correlated features can make coefficient estimates unstable and reduce interpretability.
- It may also underfit the data when relationships between features and target are complex, making it less accurate than advanced models like Random Forest.

3. Naive Bayes (GaussianNB)

Naive Bayes is a probabilistic classification algorithm based on Bayes' Theorem. It assumes that all input features are independent of each other. In this project, it predicts house price categories based on features like size, condition, and number of rooms. GaussianNB assumes that the data follows a normal distribution. It is fast, efficient, and works well with high-dimensional data. However, housing features are often correlated, violating the independence assumption. This reduces model accuracy. The assumption of normal distribution may not hold true for housing prices. It may produce oversimplified predictions.

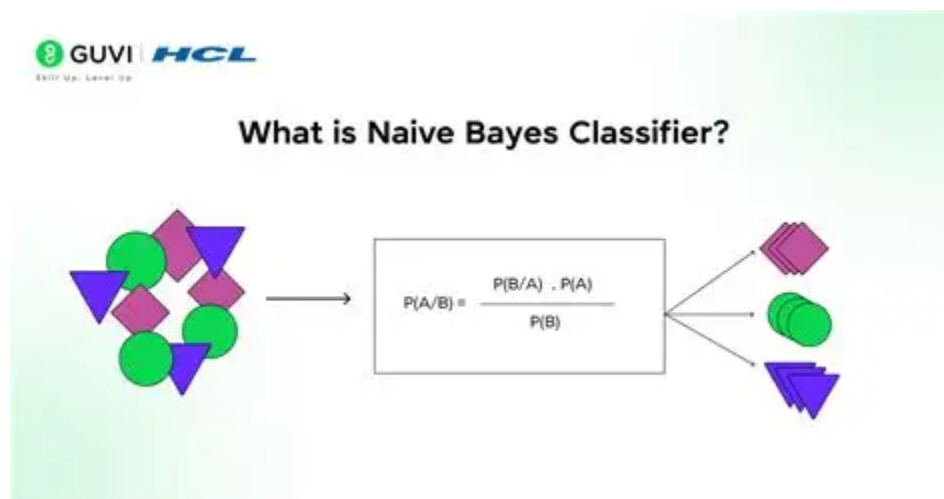


Figure 1.3: Explaining Naive Bayes (GaussianNB)

Limitations

- Naive Bayes relies on the strong assumption that all features are **independent**, which is rarely true in real-world datasets. This can reduce the model's effectiveness when features are correlated.
- GaussianNB assumes that the data follows a **normal (Gaussian) distribution**, which may not always hold true, especially in skewed datasets.
- The model may produce **lower accuracy** compared to more sophisticated algorithms when feature interactions are important.
- It can be sensitive to how data is distributed and may struggle with **imbalanced datasets**.
- Although it is fast and efficient, its simplicity can sometimes lead to **oversimplified predictions** that do not capture complex patterns.

4. Decision Tree Classifier

Decision Tree is a supervised learning algorithm that splits data into branches based on feature values. It creates rules such as “if square footage is high, then price category is high.” In this project, it helps model non-linear relationships in housing data. It is easy to understand and visualize like a flowchart. It can handle both numerical and categorical data effectively. However, it is prone to overfitting, especially with complex housing features. Small changes in data can lead to different tree structures. It may not generalize well to new data. Deep trees can become complex and less interpretable.

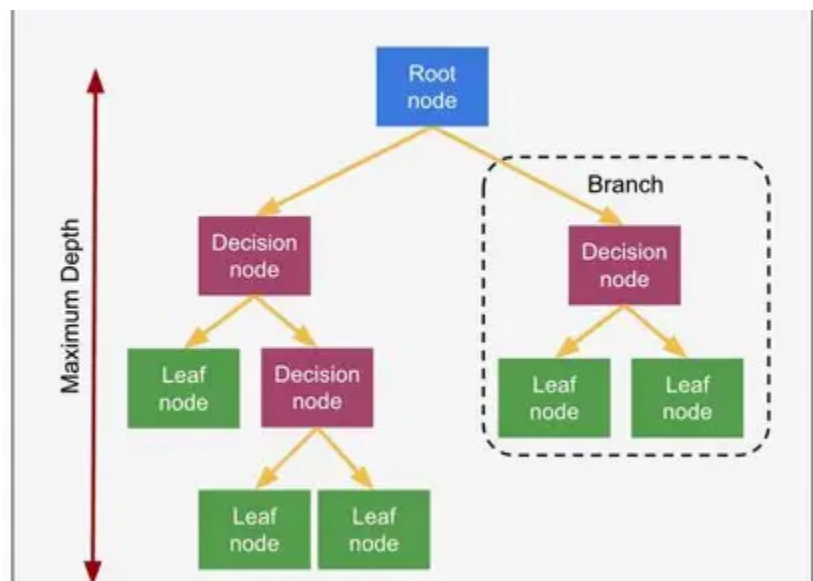


Figure 1.4: Explaining Decision Tree Classifier

Limitations

- Decision Trees are highly prone to overfitting, especially when the tree grows deep and captures noise instead of general patterns.
- They have high variance, meaning small changes in the dataset can result in completely different tree structures.
- Decision Trees may not generalize well to unseen data unless techniques like pruning or depth limitation are applied.
- They tend to create biased trees if the dataset is imbalanced, favoring dominant classes.
- Additionally, with many features, trees can become overly complex and harder to interpret, reducing their practical usability.

5. Random Forest Classifier

Random Forest is an ensemble learning algorithm that combines multiple decision trees. Each tree is trained on different subsets of housing data and features. The final prediction is made using majority voting. It improves accuracy and reduces overfitting compared to a single decision tree. In this project, it captures complex relationships between features like area, grade, and price. It is robust and performs well on large datasets. However, it requires more computational power. It is less interpretable than a single tree. It is still not ideal for continuous price prediction.

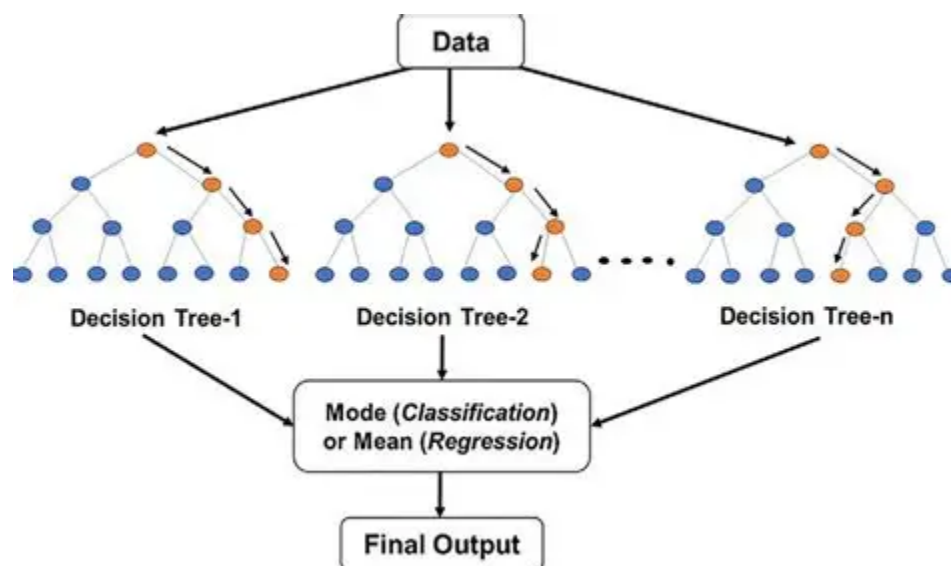


Figure 1.5: Explaining Random Forest Algorithm

Limitations

- Random Forest reduces overfitting but introduces increased computational complexity, as it builds multiple decision trees instead of one.
- Training can be slower, and the model requires more memory and processing power, especially with a large number of trees.
- It is less interpretable compared to a single Decision Tree, making it difficult to understand how predictions are made.
- Although robust, it may not perform optimally on very small datasets, where simpler models could be more effective.
- Proper tuning of hyperparameters like the number of trees, depth, and features is necessary, otherwise the model may become inefficient or unnecessarily complex.

K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a distance-based classification algorithm. It predicts the output based on the majority class of the nearest data points. In this project, it compares a house with similar houses using features like area and number of rooms. It uses distance metrics such as Euclidean distance to measure similarity. It is simple and easy to implement. It works well when similar data points exist in the dataset. However, it is slow for large datasets like KC housing data. It is sensitive to feature scaling and outliers. Performance decreases with high-dimensional data.

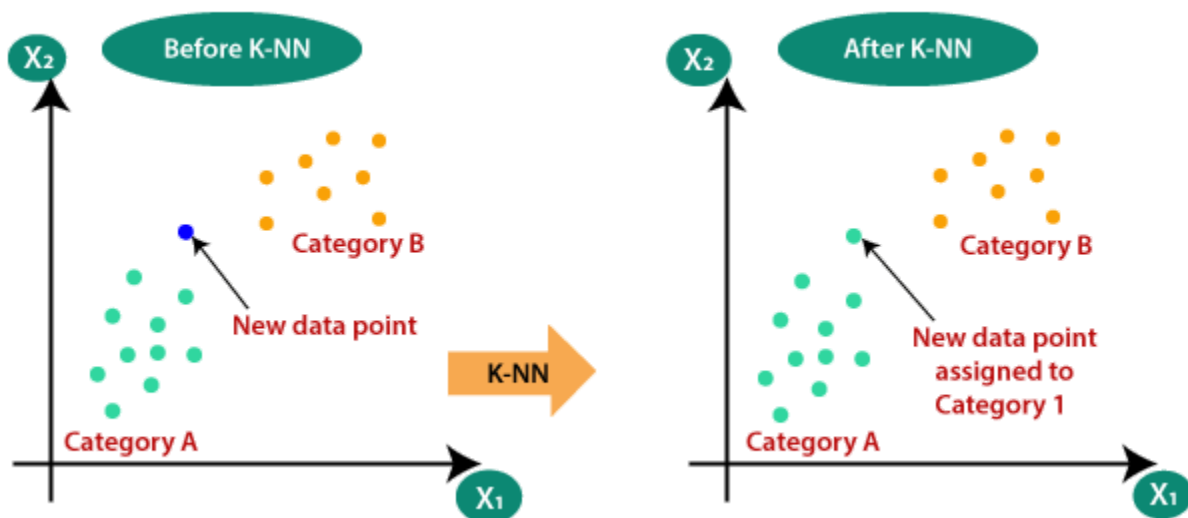


Figure 1.6: Explaining KNN

Limitations

KNN has several limitations that can affect its performance in real-world scenarios:

- **High Computational Cost**

KNN is slow during prediction because it calculates the distance between the test point and all training data points. This becomes inefficient for large datasets.

- **Choice of K Value**

Selecting the optimal K is difficult.

- Small K → sensitive to noise (overfitting)
- Large K → less accurate (underfitting)

- **Curse of Dimensionality**

As the number of features increases, the distance between points becomes less meaningful, reducing model performance.

- **Sensitive to Feature Scaling**

Features with larger values dominate distance calculations. Without normalization (like StandardScaler), results become biased.

- **Affected by Noise and Outliers**

Noisy data or outliers can mislead the algorithm, especially when K is small.

- **Memory Intensive**

Since it stores all training data, it requires more memory compared to other models.

Gradient Boosting Classifier

Gradient Boosting is an ensemble learning algorithm that builds models sequentially. Each new model corrects the errors made by previous models. It combines multiple weak learners, usually decision trees, to improve performance. In this project, it captures complex relationships between housing features and price categories. It provides high accuracy and strong predictive performance. It is widely used for structured data problems. However, training is slow due to its sequential nature. It requires careful tuning of parameters. It is sensitive to noise and outliers. Overfitting may occur if too many models are built.

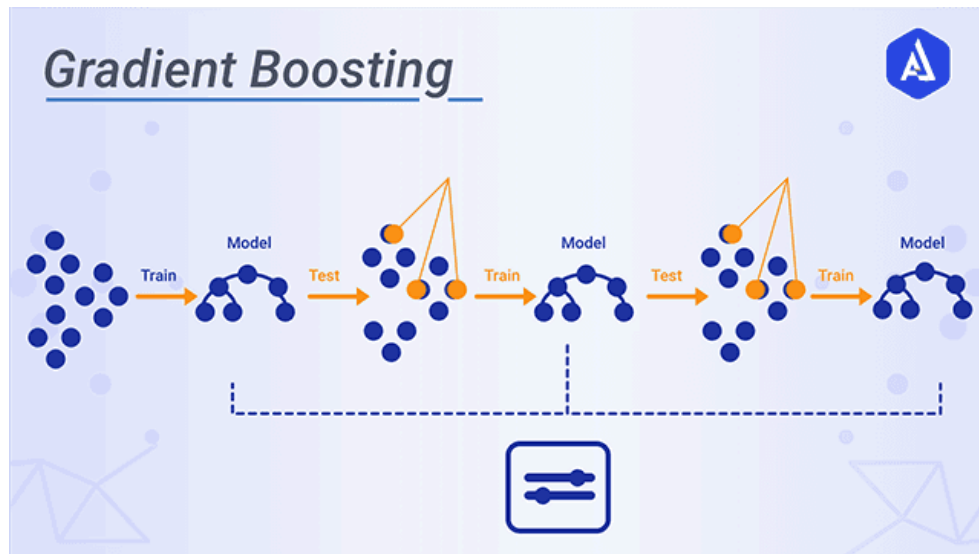


Figure 1.7: Elaborating Gradient Boosting

Limitations

Despite its high performance, Gradient Boosting has several drawbacks:

- **High Computational Complexity**
Training is slow because models are built sequentially. Each step depends on the previous one, making it time-consuming.
- **Requires Careful Hyperparameter Tuning**
Parameters like:
 - Learning rate
 - Number of trees
 - Tree depthmust be tuned properly. Poor tuning can reduce performance.
- **Risk of Overfitting**
If too many trees are added or the learning rate is too high, the model may memorize training data instead of generalizing.
- **Sensitive to Noise and Outliers**
Since each model tries to correct previous errors, noisy data can get amplified, affecting accuracy.
- **Less Interpretability**
The model is complex and difficult to interpret compared to simpler models like Decision Trees.
- **Sequential Nature (No Parallelization)**
Unlike Random Forest, Gradient Boosting cannot be easily parallelized, increasing training time.

Conclusion

In this project, multiple machine learning algorithms were applied to analyze housing data and compare their performance.

Each algorithm uses a different approach, such as linear models, probability-based methods, and ensemble techniques.

Ensemble models like Random Forest and Gradient Boosting performed better due to their ability to capture complex relationships.

Simple models like Logistic Regression and Naive Bayes showed limitations with non-linear housing data.

Since house price is a continuous variable, regression models would be more suitable for accurate prediction.

Regression Algorithms Implemented in the Project

Linear Regression

Linear Regression is a basic regression algorithm that models the relationship between input features and a continuous target variable. In this project, it predicts house prices based on features such as bedrooms, square footage, and condition. It assumes a linear relationship between features and price. The model is simple, fast, and easy to interpret, making it useful as a baseline model. However, housing data often contains non-linear relationships that Linear Regression cannot capture effectively. It may underfit complex data. It is also sensitive to outliers, which can affect prediction accuracy.

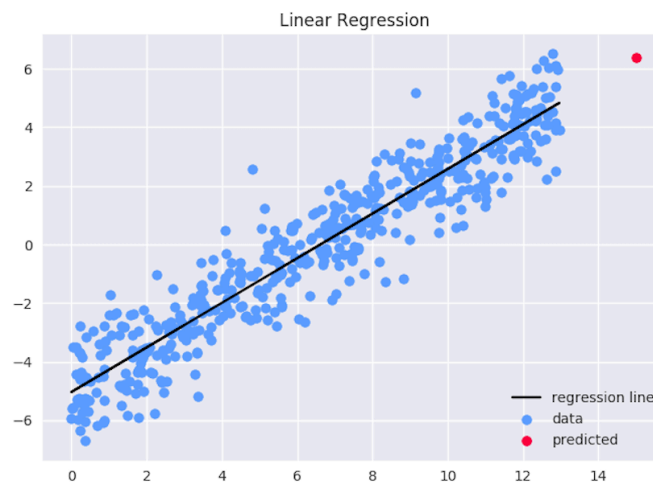


Figure 1.8: Linear Regression

Limitations

- Linear Regression assumes a linear relationship, which may not hold true for housing data.
- It is sensitive to outliers and can give inaccurate predictions.
- It cannot capture complex feature interactions.
- It may underfit when data is non-linear.

Ridge Regression

Ridge Regression is an improved version of Linear Regression that adds regularization to reduce overfitting. It penalizes large coefficients and helps stabilize the model. In this project, it is used to predict house prices while handling correlated features like `sqft_living` and `sqft_lot`. It improves generalization compared to Linear Regression. It is useful when multiple features influence price. However, it still assumes a linear relationship and may not capture complex patterns in housing data.

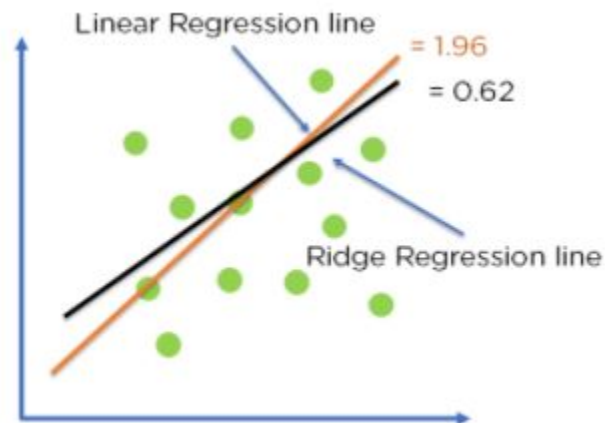


Figure 1.9: Linear Regression vs Ridge Regression

Limitations

- It cannot handle strong non-linear relationships.
- It does not perform feature selection (keeps all features).
- Performance depends on tuning the regularization parameter.
- May still underfit complex housing data.

Decision Tree Regressor

Decision Tree Regressor is a tree-based algorithm that predicts continuous values by splitting data into regions. It creates rules based on features like area and condition to estimate house prices. In this project, it captures non-linear relationships effectively. It is easy to interpret and visualize. It works well with structured data. However, it is prone to overfitting and may create overly complex trees. Small changes in data can lead to different results.

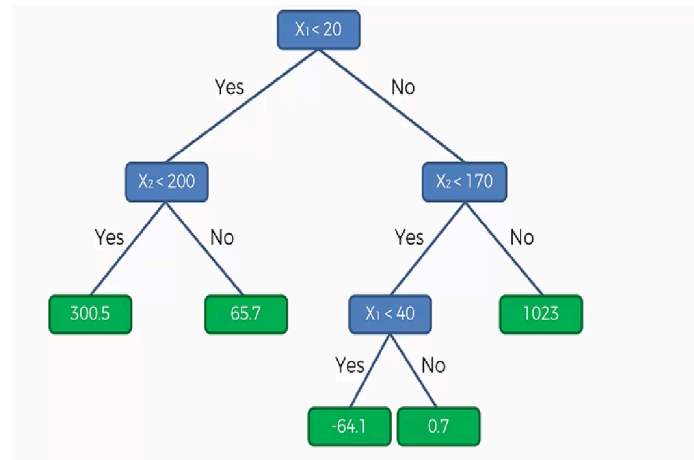


Figure 1.10: Decision Tree Regression

Limitations

- Highly prone to overfitting if not controlled.
- Sensitive to small changes in data.
- May not generalize well to unseen data.
- Requires pruning or tuning for better performance.

Random Forest Regressor

Random Forest Regressor is an ensemble method that combines multiple decision trees to improve prediction accuracy. Each tree is trained on different subsets of data and features. In this project, it predicts house prices more accurately by capturing complex relationships. It reduces overfitting compared to a single decision tree. It performs well on large datasets like KC housing data. However, it requires more computational resources and is less interpretable.

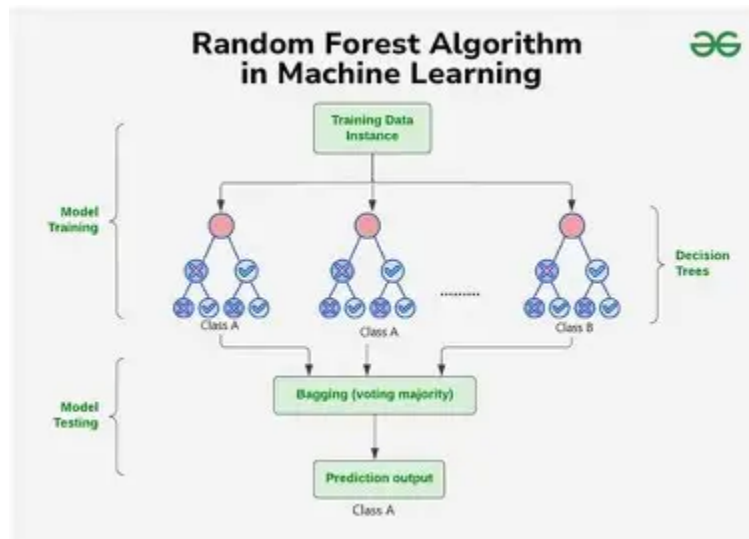


Figure 1.11: Random Forest Algorithm

Limitations

- Requires more memory and computation.
- Less interpretable compared to simple models.
- Training can be slower with many trees.
- Needs parameter tuning for best performance.

Support Vector Regressor (SVR)

Support Vector Regressor (SVR) is the regression version of SVM that predicts continuous values. It tries to fit the best line within a margin of tolerance. In this project, it models relationships between housing features and price. It works well for smaller datasets and complex patterns. It uses kernel functions to handle non-linear relationships. However, it is computationally expensive and not suitable for very large datasets.

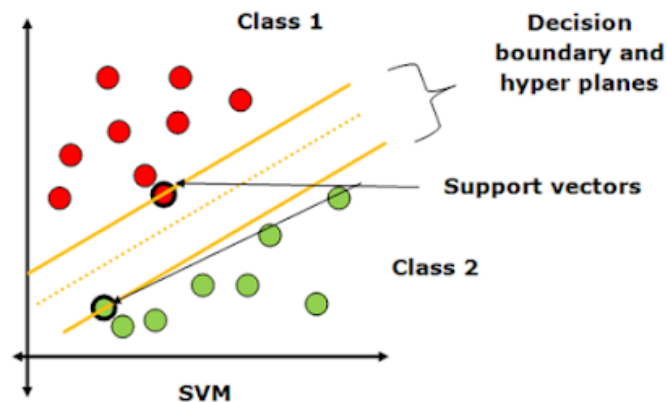


Figure 1.12: Support Vector Regressor

Limitations

- Slow for large datasets.
- Requires careful parameter tuning.
- Sensitive to feature scaling.
- Performance decreases with large and noisy data.

Gradient Boosting Regressor

Gradient Boosting Regressor is an ensemble algorithm that builds models sequentially to reduce prediction errors. Each new model improves upon the previous one. In this project, it predicts house prices by capturing complex feature interactions. It provides high accuracy and strong performance. It is widely used in real-world applications. However, it requires careful tuning and is computationally expensive.

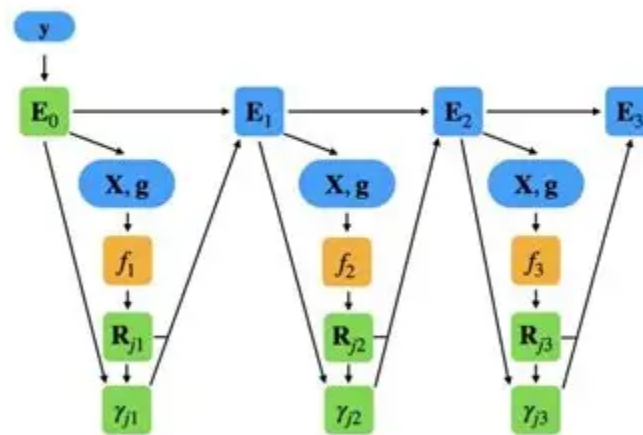


Figure 1.13: Gradient Boosting Regressor

Limitations

- Training is slow due to sequential learning.
- Sensitive to noise and outliers.
- Requires careful hyperparameter tuning.
- Can overfit if too many trees are used.

Conclusion

In this section, regression algorithms were applied to predict house prices using the KC housing dataset.

Models like Linear Regression provided a simple baseline, while ensemble methods such as Random Forest and Gradient Boosting delivered better performance.

Tree-based models handled non-linear relationships effectively compared to linear models.

Among all, ensemble regressors showed higher accuracy due to their ability to capture complex patterns.

Overall, regression algorithms proved more suitable for this dataset as they predict continuous values like house prices more accurately.

Final Conclusion

This project demonstrates a complete machine learning workflow, including data preprocessing, feature scaling, model training, and evaluation using the KC housing dataset.

Both classification and regression algorithms were applied to understand and compare their performance on real-world data.

Classification models helped in learning different approaches, while regression models provided more accurate predictions for house prices.

Among all models, ensemble methods like Random Forest and Gradient Boosting showed better performance due to their ability to capture complex relationships.

Since house price is a continuous variable, regression algorithms are more suitable and effective for practical prediction tasks.

What we learned:

Through this project, we gained practical knowledge of applying both classification and regression algorithms to real-world housing data.

We learned how to load, preprocess, and scale data using tools like Pandas and StandardScaler.

We understood how different models work and how their performance varies based on data characteristics.

We also learned to train models, make predictions, and evaluate results using both accuracy (classification) and R^2 score (regression).

Additionally, we realized the importance of choosing the right approach, as regression is more suitable for predicting continuous values like house prices.